

SENIOR BLOCKCHAIN EXPERT

20-Day Intensive Course

20 complete 60-minute lectures for LinkedIn, Upwork, interviews, and portfolio development

Solidity | EVM | Foundry | DeFi | Security | L2 | Cross-Chain | Solana | Architecture

Course philosophy: Twenty lectures can build a rigorous senior-level map and strong portfolio foundation, but senior expertise also requires sustained implementation, debugging, security review, and production ownership. Every day in this course therefore ends with a concrete deliverable.

Prepared August 2026

Reference toolchain aligned to current official Solidity, Foundry, OpenZeppelin, and Solana documentation.

How to Use This Course

Each day is designed for approximately 60 minutes of instructor-led learning. The minute-by-minute agenda is a pacing guide, not a strict timer; difficult labs can continue as homework.

For maximum job value, do the code and portfolio deliverables. Reading alone will not produce the debugging instincts expected from a senior engineer.

Use testnets, local nodes, forks, and deliberately small amounts. Never use unaudited course contracts with meaningful real funds.

Keep one GitHub monorepo for daily exercises and separate polished repositories for the strongest portfolio projects.

Course Outcomes

- Build and test Solidity contracts using modern EVM tooling.
- Integrate ERC standards, DEXs, lending protocols, oracles, proxies, backends, and indexers.
- Reason about security with threat models, fuzz/invariant tests, and audit-style findings.
- Explain L2, cross-chain, and Solana architecture during interviews.
- Design production systems with multisig, timelocks, monitoring, incident response, and deployment controls.
- Present credible blockchain engineering work to LinkedIn recruiters and Upwork clients.

Clickable Course Contents

[How to Use This Course](#)

[Day 1: Blockchain Architecture](#)

[Day 2: Ethereum & EVM Internals](#)

[Day 3: Solidity Fundamentals](#)

[Day 4: Advanced Solidity](#)

[Day 5: ERC Standards & OpenZeppelin](#)

[Day 6: Foundry & Professional Testing](#)

[Day 7: Smart Contract Security I](#)

[Day 8: Security II & Auditing](#)

[Day 9: DeFi & AMMs](#)

[Day 10: Lending, Flash Loans & Arbitrage](#)

[Day 11: Oracles & Data Feeds](#)

[Day 12: Upgradeable Contracts](#)

[Day 13: Web3 Backend Engineering](#)

[Day 14: Indexing & Analytics](#)

[Day 15: Layer 2 & Rollups](#)

[Day 16: Cross-Chain & Bridge Security](#)

[Day 17: Solana, Rust & Anchor](#)

[Day 18: Senior System Architecture](#)

[Day 19: DeFi Vault Capstone](#)

[Day 20: Career, Upwork & Interviews](#)

[Appendix A: Interview Bank](#)

[Appendix B: Portfolio Checklist](#)

[References](#)

Day 1 - Blockchain Architecture, Cryptography, and Transaction Lifecycle

Lecture goal: Build a senior-level mental model of how blockchains move from signed user intent to globally agreed state.

Portfolio output: A polished transaction lifecycle explanation and architecture diagram suitable for a portfolio README.

Learning Outcomes

- Explain blocks, transactions, state, nodes, validators, consensus, finality, and reorgs using precise engineering language.
- Distinguish hashing, encryption, digital signatures, public keys, private keys, addresses, and Merkle proofs.
- Compare Bitcoin UTXO architecture with Ethereum account/state architecture.
- Trace an Ethereum transaction from wallet signing through RPC submission, mempool propagation, block inclusion, execution, receipt, and finality.

60-Minute Teaching Plan

Time	Focus
0-5 min	Course framing and senior-engineer mindset
5-15 min	Distributed ledgers and state machines
15-25 min	Cryptography and ownership
25-35 min	Bitcoin vs Ethereum execution models
35-45 min	Consensus, blocks, reorgs, and finality
45-55 min	Transaction-lifecycle lab
55-60 min	Knowledge check and homework

1.1 Blockchain as a replicated state machine

A blockchain is best understood as a replicated state machine rather than merely a database. Every full node begins from an agreed history and independently applies valid state transitions. The engineering value comes from deterministic execution plus a consensus protocol that decides which ordered history is canonical. Separate the data layer (transactions and blocks), execution layer (how transactions change state), consensus layer (how participants agree on ordering), networking layer (peer discovery and propagation), and application layer (wallets, dApps, exchanges, indexers). A senior engineer should always ask which layer owns a given guarantee. For example, a wallet can sign a transaction but cannot guarantee inclusion; a validator can propose a block but cannot unilaterally define long-term finality; an RPC provider can relay a request but may censor, lag, or return stale data.

1.2 Cryptography and account ownership

Private keys are secret numbers used to create digital signatures. Public keys allow other participants to verify those signatures without learning the secret key. Hash functions map arbitrary input to fixed-size output and are

designed to make preimage discovery and collisions computationally impractical. Ethereum addresses are derived from public-key material, while transaction authenticity comes from signatures. Explain the difference between confidentiality and authenticity: normal blockchain transactions are usually public, so signatures prove authorization rather than hide content. Merkle trees compress large sets of data into a root commitment, making inclusion proofs efficient. In interviews, avoid saying a blockchain "encrypts everything"; most public-chain data is not encrypted.

1.3 UTXO vs account-based state

Bitcoin models spendable value as unspent transaction outputs. A transaction consumes previous outputs and creates new outputs. Ethereum uses accounts with balances, nonces, code, and storage. Externally owned accounts are controlled by private keys, while contract accounts are controlled by deployed code. The account model makes general-purpose stateful programs natural, but it also introduces issues such as nonce ordering, reentrancy between contracts, and shared mutable state. Understanding both models helps when evaluating protocol designs, bridges, wallet behavior, and cross-chain differences.

1.4 Consensus, reorgs, and finality

Consensus is not the same thing as execution. Execution answers "what does this transaction do?" while consensus answers "which ordered block history should nodes accept?" Proof-of-stake systems use validators and economic penalties to secure agreement. Distinguish block proposal, confirmation, probabilistic safety, and stronger forms of finality. A chain reorganization can replace recently accepted blocks, so production systems often wait for additional confirmations or finalized checkpoints before treating deposits as irreversible. This becomes critical for exchanges, bridges, liquidation bots, and payment processors.

1.5 Transaction lifecycle

A wallet constructs a transaction containing destination, value, calldata, nonce, chain ID, and fee parameters. The wallet signs the transaction locally, producing a signed payload. An RPC node receives the raw transaction, validates basic rules, and propagates it to peers. Block builders or validators select transactions, order them, and execute them against current state. Successful or reverted execution consumes gas; the resulting state root, logs, receipt, and block metadata become observable through RPC. Senior developers distinguish "broadcast", "pending", "included", "confirmed", and "finalized" states because each has different application semantics.

Code / Architecture Example

Transaction lifecycle

```
Wallet -> Sign -> RPC -> Peer network -> Builder/Validator
      -> Block inclusion -> EVM execution -> Receipt -> Confirmations -> Finality
```

Security boundary:

```
Private key should remain in wallet/HSM; RPC providers should never need it.
```

Hands-On Lab (45-55 minutes)

1. Install Git, VS Code, Node.js, Python, Foundry, and a browser wallet on a test profile.
2. Use a block explorer to open one successful ETH transfer and identify from, to, nonce, gas used, effective gas price, block number, status, and confirmations.
3. Open one contract interaction and locate the method selector, decoded arguments, event logs, and internal calls.
4. Write a one-page explanation titled "What happens after I click Send?" without using vague phrases such as "the blockchain verifies it".

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?

- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: What is the difference between hashing and encryption?

A: Hashing is one-way commitment/fingerprinting; encryption is reversible with the correct key and provides confidentiality.

Q: What is a reorg?

A: Replacement of a previously accepted recent chain segment by another valid chain that wins according to consensus fork-choice rules.

Q: Why does an Ethereum account have a nonce?

A: It orders transactions from an EOA and prevents replay of the same signed transaction on the same chain context.

Homework

Create GitHub repository `senior-blockchain-20day` with `/day01/README.md`, a transaction-lifecycle diagram, and a glossary of 30 terms.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 2 - Ethereum, EVM Internals, ABI, Storage, and Gas

Lecture goal: Understand how Solidity becomes EVM execution and how data location, storage layout, calldata, and gas affect design.

Portfolio output: A small EVM/RPC inspector project with commands and annotated outputs.

Learning Outcomes

- Explain stack, memory, storage, calldata, returndata, logs, and bytecode.
- Decode function selectors and ABI-encoded arguments.
- Read simple Solidity storage layouts and reason about gas-heavy operations.
- Use RPC and Foundry Cast to inspect live/test-chain state.

60-Minute Teaching Plan

Time	Focus
0-5 min	Review
5-15 min	EVM execution model
15-25 min	ABI and calldata
25-35 min	Storage layout and data locations
35-45 min	Gas accounting and transaction fees
45-55 min	Cast/RPC lab
55-60 min	Quiz

2.1 EVM execution model

The EVM is a deterministic stack-based virtual machine. Contract bytecode is executed as opcodes operating primarily on a 256-bit word stack. Memory is temporary, byte-addressable, and cleared between calls. Storage is persistent key-value state attached to a contract address and is comparatively expensive. Calldata contains read-only input supplied to an external call. Returndata contains output from the most recent external call. Logs are append-only receipt data used by off-chain systems; contracts cannot later read historical logs. Senior engineers use these distinctions when optimizing gas, designing interfaces, and reviewing security-critical low-level calls.

2.2 ABI and function selectors

The ABI defines how typed function calls and return values are encoded. An external function call begins with a 4-byte selector derived from the function signature, followed by ABI-encoded parameters. Static types occupy fixed 32-byte slots; dynamic types use offsets to separate the head and tail portions. Events use topics and data, with the event signature commonly stored in topic0 for non-anonymous events. ABI knowledge is essential when debugging raw transactions, writing bots, decoding revert data, using proxies, or integrating contracts from Python/TypeScript.

2.3 Storage layout

Solidity packs some state variables into 32-byte storage slots when types fit and ordering permits. Mappings and dynamic arrays use deterministic hash-derived locations rather than storing all members inline. Proxy systems

make storage layout especially important because the proxy owns the state while implementation code interprets it. Reordering, deleting, or changing types can corrupt existing state during upgrades. Learn to inspect compiler-generated storage layout instead of guessing.

2.4 Gas and fees

Gas measures computational/state-resource consumption, while the amount paid depends on gas used and effective gas price. Persistent storage writes, contract creation, and some external-call patterns are expensive relative to arithmetic in memory. Refund rules and opcode pricing can change across hard forks, so avoid relying on folklore numbers. Optimize only after correctness and security: bad gas micro-optimizations can make code unreadable or unsafe. For user-facing systems, estimate gas, handle fee spikes, expose sensible slippage/deadline controls, and understand how reverts still consume gas.

2.5 RPC as the application boundary

Most dApps do not talk directly to validators; they talk to JSON-RPC endpoints. `eth_call`` executes locally without publishing a transaction, while `eth_sendRawTransaction`` broadcasts a signed payload. Other calls retrieve blocks, receipts, logs, code, balances, and storage. RPC providers can rate-limit, lag, disagree briefly, or omit pending-state details, so production architectures use retries, multiple providers, or self-hosted nodes for critical workloads.

Code / Architecture Example

```
// Function selector concept
keccak256("transfer(address,uint256)")[:4]

// Useful Cast commands
cast balance <address> --rpc-url $RPC_URL
cast code <contract> --rpc-url $RPC_URL
cast call <contract> "balanceOf(address)(uint256)" <user> --rpc-url $RPC_URL
cast receipt <tx_hash> --rpc-url $RPC_URL
cast storage <contract> 0 --rpc-url $RPC_URL
```

Hands-On Lab (45-55 minutes)

5. Start Anvil and deploy a tiny storage contract.
6. Use `cast storage`` before and after updating a state variable.
7. Encode and decode a function call using `cast calldata`` and `cast abi-decode``.
8. Compare `eth_call`` with sending a signed state-changing transaction.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Storage vs memory?

A: Storage persists between transactions and belongs to contract state; memory is temporary per call and cheaper for transient data.

Q: What is calldata?

A: Read-only call input supplied to an external function; it avoids copying when parameters can be processed directly.

Q: Can a contract read old event logs?

A: No. Logs are for transaction receipts/off-chain consumers; contracts cannot query historical logs through normal EVM opcodes.

Homework

Document five common RPC methods, their purpose, and whether they mutate chain state.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 3 - Solidity Fundamentals - Types, Functions, Errors, Events, and Safe Value Transfer

Lecture goal: Write clean, testable Solidity with explicit state transitions and predictable error handling.

Portfolio output: SimpleBank contract, tests, and README documenting invariants.

Learning Outcomes

- Use Solidity types, visibility, mutability, constructors, constants, immutables, structs, enums, arrays, and mappings.
- Design events and custom errors for production observability.
- Explain ``msg.sender``, ``msg.value``, ``block.timestamp``, and receive/fallback behavior.
- Build a SimpleBank contract using checks-effects-interactions.

60-Minute Teaching Plan

Time	Focus
0-5 min	Syntax and compiler mindset
5-15 min	Types and state
15-25 min	Functions and visibility
25-35 min	Errors and events
35-45 min	ETH transfer patterns
45-55 min	SimpleBank lab
55-60 min	Review

3.1 Solidity state and types

Solidity is statically typed. State variables become part of persistent contract storage; local variables normally live in stack or memory depending on type and compiler decisions. Value types include integers, booleans, addresses, enums, and fixed-size bytes. Reference types include arrays, strings, mappings, structs, and bytes. Know where each variable lives and whether it is copied or referenced. Use ``constant`` for compile-time constants and ``immutable`` for constructor-set values that never change afterward.

3.2 Function API design

Visibility expresses who can invoke a function: ``external``, ``public``, ``internal``, or ``private``. State mutability such as ``view``, ``pure``, and ``payable`` communicates behavioral constraints. Prefer small functions with clear preconditions and minimal privilege. External interfaces should be stable and intentional because every public function is part of the protocol attack surface. Consider whether a function should support contracts as callers, whether it can be front-run, and whether its return values are useful to integrations.

3.3 Errors and events

Use reverts to protect invariants. Custom errors are expressive and often cheaper than long revert strings. `require` remains useful for concise preconditions, while `assert` should represent conditions that should be impossible if the program is correct. Events create an off-chain audit trail for indexers, dashboards, accounting, and monitoring. Emit events after meaningful state changes and include enough indexed fields to support expected queries.

3.4 ETH transfer and external calls

Direct ETH transfers are external interactions. Modern contracts typically use low-level `call{value: amount}("")` and check the returned success flag rather than relying on historical gas-stipend assumptions. Any external call can transfer control to untrusted code. Therefore, update internal accounting before the external call when possible, use reentrancy protection where appropriate, and keep external-call surfaces minimal.

3.5 Contract design discipline

A professional contract separates authorization, validation, state mutation, and external interaction. Name units clearly, avoid magic numbers, document trust assumptions, and emit events that let an operator reconstruct important actions. Begin thinking in invariants: total recorded user balances must never exceed actual ETH controlled by the bank; a withdrawal must reduce the user balance exactly once; unauthorized users must not move another user's funds.

Code / Architecture Example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

contract SimpleBank {
    mapping(address => uint256) public balances;
    error InsufficientBalance();
    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);

    function deposit() external payable {
        require(msg.value > 0, "zero value");
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) external {
        if (balances[msg.sender] < amount) revert InsufficientBalance();
        balances[msg.sender] -= amount;
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok, "transfer failed");
        emit Withdrawn(msg.sender, amount);
    }
}
```

Hands-On Lab (45-55 minutes)

9. Create Foundry project `day03-simple-bank`.
10. Implement deposit, partial withdrawal, full withdrawal, and a read-only balance function.
11. Add custom errors and events.
12. Write at least four unit tests, including a revert case.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?

- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why are external calls dangerous?

A: They can fail, reenter, consume unexpected gas, or invoke arbitrary recipient code.

Q: When would you use immutable?

A: For values set at deployment time that must never change, such as a fixed dependency address or configuration value.

Q: Why emit events?

A: They provide efficient off-chain observability for indexers, UIs, monitoring, and audit trails.

Homework

Extend SimpleBank with an owner-controlled pause that blocks deposits/withdrawals without allowing the owner to seize user balances.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 4 - Advanced Solidity - Inheritance, Interfaces, Libraries, Low-Level Calls, and Assembly

Lecture goal: Understand the mechanisms that underpin real protocol composition, proxies, routers, and reusable libraries.

Portfolio output: Treasury contract plus low-level-call architecture notes.

Learning Outcomes

- Use interfaces, abstract contracts, inheritance, libraries, modifiers, and custom types appropriately.
- Explain ``call``, ``delegatecall``, and ``staticcall`` including context differences.
- Understand fallback/receive execution.
- Read basic Yul/assembly without treating it as magic.

60-Minute Teaching Plan

Time	Focus
0-5 min	Design review
5-15 min	Inheritance and interfaces
15-25 min	Libraries and modifiers
25-35 min	Fallback/receive and low-level calls
35-45 min	delegatecall and context
45-55 min	Treasury lab
55-60 min	Review

4.1 Interfaces and inheritance

Interfaces define externally callable behavior without implementation and allow contracts to integrate against standards or third-party protocols. Abstract contracts can contain partial implementation and reusable state. Inheritance should express a genuine "is-a" relationship or reusable module; deep inheritance trees make security review harder. Senior developers inspect method resolution, overridden behavior, constructors/initializers, and storage introduced by base contracts.

4.2 Libraries and modifiers

Libraries package reusable logic. Internal library functions can be inlined by the compiler, while external/public library calls may involve deployed library code depending on design. Modifiers can centralize access checks but should stay simple; overly complex modifiers hide control flow and complicate audits. Prefer explicit helper functions when a modifier performs more than guard logic.

4.3 receive, fallback, and low-level calls

``receive()`` handles empty-calldata ETH transfers when defined. ``fallback()`` runs when no function selector matches or when calldata is otherwise unhandled. Low-level ``call`` returns ``(success, returndata)`` instead of

automatically bubbling typed failures. ``staticcall`` forbids state changes in the called context. Low-level calls are sometimes necessary for generic routers, proxies, token compatibility, and plugin systems, but they reduce compiler guarantees.

4.4 delegatecall

``delegatecall`` executes code from another address while preserving the caller contract's storage, address context, and ``msg.sender`/`msg.value`` behavior. This is the foundation of many proxy architectures and also one of the most dangerous EVM features. The implementation code interprets the calling contract's storage, so storage layout must match expectations. Never `delegatecall` arbitrary user-supplied targets in a privileged contract.

4.5 Assembly mindset

Inline assembly/Yul allows direct EVM operations and can improve performance or implement low-level patterns unavailable in high-level Solidity. It also bypasses type safety and compiler checks. Learn to read ``mload``, ``mstore``, ``calldataload``, ``sload``, ``sstore``, and ``revert``. Use assembly only when the benefit is concrete and well tested.

Code / Architecture Example

```
interface IERC20Like {
    function transfer(address to, uint256 amount) external returns (bool);
}

function rawCall(address target, bytes calldata data)
    external returns (bytes memory result)
{
    (bool ok, bytes memory ret) = target.call(data);
    if (!ok) assembly {
        revert(add(ret, 32), mload(ret))
    }
    return ret;
}
```

Hands-On Lab (45-55 minutes)

13. Build a Treasury contract with Ownable-style authorization, deposit events, ERC-20 rescue, ETH withdrawal, and pause.
14. Create an interface and use it to transfer a test ERC-20.
15. Add one raw call helper only for an allowlisted target and document why arbitrary calls would be unsafe.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: call vs delegatecall?

A: call executes target code in target storage/address context; delegatecall executes target code in caller storage/address context.

Q: receive vs fallback?

A: receive handles empty calldata ETH receipts; fallback catches unmatched calls and may also receive ETH if payable.

Q: Why can inheritance be risky?

A: Complex method resolution, hidden state, initializer order, and overridden behavior can make control flow and storage assumptions difficult to audit.

Homework

Draw the storage/context differences among CALL, DELEGATECALL, and STATICCALL.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 5 - ERC Standards, OpenZeppelin, Token Design, Permit, and Role-Based Access

Lecture goal: Build token systems using battle-tested standards and explicit governance rather than ad-hoc code.

Portfolio output: Production-style ERC-20 repository with tests and governance notes.

Learning Outcomes

- Explain ERC-20 allowance mechanics and common pitfalls.
- Compare ERC-20, ERC-721, and ERC-1155 use cases.
- Use OpenZeppelin Contracts for access control and token extensions.
- Understand EIP-712 typed data and ERC-2612-style permit at a conceptual/implementation level.

60-Minute Teaching Plan

Time	Focus
0-5 min	Standards mindset
5-15 min	ERC-20 internals
15-25 min	NFT/multi-token standards
25-35 min	OpenZeppelin composition
35-45 min	Roles, permit, signatures
45-55 min	Production token lab
55-60 min	Review

5.1 Why standards matter

Standards create interoperability among wallets, exchanges, protocols, indexers, and tools. A token that approximately behaves like ERC-20 but violates return values or event expectations can break integrations. Senior engineers read the specification, use audited libraries, and test edge cases rather than copying a minimal tutorial implementation.

5.2 ERC-20 mechanics

ERC-20 tracks balances and allowances. `approve` authorizes a spender, and `transferFrom` lets that spender move tokens up to the allowance. Understand allowance race concerns, infinite approvals, fee-on-transfer/nonstandard tokens, tokens that do not return booleans, and how protocols use safe wrappers. Supply policy is a separate design decision: fixed supply, mintable supply, capped supply, burning, rebasing, or externally controlled issuance each creates different risk.

5.3 NFTs and multi-token systems

ERC-721 models individually distinguishable token IDs with ownership and approvals. ERC-1155 can represent many fungible or non-fungible token IDs in one contract and supports efficient batch operations. Metadata is

usually referenced off-chain, so immutable content, IPFS/Arweave, centralized metadata servers, and upgradeable metadata each have different trust tradeoffs.

5.4 OpenZeppelin composition

OpenZeppelin Contracts provides reusable implementations for tokens, ownership, access control, pausing, reentrancy guards, cryptography, and proxies. Composition is safer than rewriting mature primitives, but you still own configuration risk. Review inherited functions, role admin relationships, hooks, and version changes. Pin dependencies in production builds and audit upgrade paths.

5.5 Roles and signature approvals

Role-based access control separates responsibilities such as MINTER_ROLE, PAUSER_ROLE, and UPGRADER_ROLE. Avoid concentrating all power in a single hot wallet. EIP-712 defines typed structured signing so users can sign human-meaningful domain-separated data. Permit patterns allow an allowance to be authorized by signature, reducing the need for a separate approval transaction in compatible flows. Always include nonces and domain separation to prevent replay.

Code / Architecture Example

```
contract ProfessionalToken is ERC20, ERC20Burnable, Pausable, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
    uint256 public immutable cap;

    constructor(uint256 _cap) ERC20("Portfolio Token", "PTK") {
        cap = _cap;
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MINTER_ROLE, msg.sender);
        _grantRole(PAUSER_ROLE, msg.sender);
    }
    // Add mint cap check and pause-aware transfer behavior.
}
```

Hands-On Lab (45-55 minutes)

16. Create a capped ERC-20 using OpenZeppelin.
17. Separate admin, minter, and pauser roles.
18. Add burn functionality and tests for unauthorized mint/pause.
19. Document how roles should be transferred to a multisig before production launch.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why not implement ERC-20 from scratch?

A: Mature libraries reduce implementation risk and improve standard compatibility; custom code should be justified by requirements.

Q: What is an infinite approval?

A: An allowance set to a very large/max value, common for UX but risky if the spender contract is compromised.

Q: Why use role separation?

A: It limits blast radius and supports least privilege, operational controls, and governance.

Homework

Write a token threat model covering admin key loss, compromised minter, allowance abuse, pause abuse, and supply-cap invariants.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 6 - Foundry, Hardhat, Unit Tests, Fuzzing, Invariants, Forking, and Deployment Scripts

Lecture goal: Develop the testing discipline expected of senior smart-contract engineers.

Portfolio output: Comprehensive Foundry test suite and deployment script.

Learning Outcomes

- Use Forge, Cast, Anvil, and scripts effectively.
- Write unit, fuzz, invariant, and fork tests.
- Use cheatcodes for actors, time, block state, and expectations.
- Build deterministic deployment and verification scripts.

60-Minute Teaching Plan

Time	Focus
0-5 min	Testing philosophy
5-15 min	Foundry workflow
15-25 min	Unit/fuzz tests
25-35 min	Invariant/stateful testing
35-45 min	Fork testing and impersonation
45-55 min	Token test lab
55-60 min	Coverage review

6.1 Testing pyramid for protocols

Unit tests verify individual behaviors; integration tests verify component interactions; fuzz tests explore many inputs; invariant tests explore action sequences while checking properties that must always hold; fork tests exercise code against real deployed protocol state. Production protocols use several layers because each catches different bug classes. Do not chase a coverage percentage alone - prioritize economic and authorization invariants.

6.2 Foundry workflow

Forge compiles, tests, scripts deployments, reports gas, and supports fuzz/invariant testing. Cast provides CLI interaction and encoding/decoding. Anvil runs a local EVM node with rich testing controls. Use deterministic project structure, pinned compiler/dependency versions, environment variables for RPC endpoints, and never commit private keys. Hardhat remains useful in TypeScript-heavy environments and plugin ecosystems; a senior engineer should be comfortable reading both.

6.3 Fuzz testing

A fuzz test accepts generated inputs and asserts properties for broad input ranges. Constrain invalid domains with assumptions only when they truly represent impossible inputs. Prefer asserting properties rather than specific

outputs when exploring large spaces. Common targets include token conservation, rounding behavior, fee bounds, access controls, and arithmetic at extreme values.

6.4 Invariant testing

Invariant tests generate sequences of calls and check properties after many state transitions. A handler can model realistic actions and multiple actors. Examples: protocol assets must always cover liabilities; total token supply must match sum of balances under a closed model; a user must never withdraw more shares than owned; privileged operations must only be reachable by authorized roles. Invariant testing is especially powerful for DeFi because exploits often require sequences rather than a single malformed input.

6.5 Fork testing

Fork tests create a local view of live/test chain state, letting you call actual deployed tokens, pools, routers, and oracles without risking real funds. Pin a block number for reproducibility. Test integration assumptions such as decimals, return values, liquidity, role configuration, and stale oracle behavior. Fork tests are not a replacement for isolated unit tests; they validate real-world composition.

Code / Architecture Example

```
function testFuzz_TransferConservesSupply(address to, uint96 amount) public {
    vm.assume(to != address(0));
    token.mint(address(this), amount);
    uint256 supplyBefore = token.totalSupply();
    token.transfer(to, amount);
    assertEquals(token.totalSupply(), supplyBefore);
}

function invariant_TotalSupplyNeverExceedsCap() public view {
    assertLe(token.totalSupply(), token.cap());
}
```

Hands-On Lab (45-55 minutes)

20. Add unit tests to Day 5 token.
21. Add at least two fuzz tests and one invariant test.
22. Run gas snapshot and coverage report.
23. Create a deployment script that reads config from environment and prints deployed addresses.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Fuzz vs invariant testing?

A: Fuzzing explores generated inputs to a function/property; invariant testing explores generated sequences of state-changing actions while checking persistent properties.

Q: Why pin a fork block?

A: For deterministic tests; live chain state changes constantly.

Q: Is 100% coverage enough?

A: No. Coverage shows executed lines/branches, not whether critical economic/security properties are correct.

Homework

Write a test plan before writing any new contract code: happy paths, authorization failures, boundary values, external call failures, and invariants.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 7 - Smart Contract Security I - Reentrancy, Access Control, External Calls, Front-Running, and DoS

Lecture goal: Recognize and reproduce high-impact vulnerability classes before attempting audits.

Portfolio output: Reentrancy exploit lab and professional vulnerability write-up.

Learning Outcomes

- Explain and exploit classic reentrancy in a local environment.
- Identify broken authorization and confused-deputy patterns.
- Reason about front-running/sandwich risk and unsafe deadlines/slippage.
- Use checks-effects-interactions and reentrancy guards appropriately.

60-Minute Teaching Plan

Time	Focus
0-5 min	Security mindset
5-15 min	Reentrancy
15-25 min	Authorization failures
25-35 min	External call/DoS risks
35-45 min	Front-running and transaction ordering
45-55 min	Exploit/patch lab
55-60 min	Review

7.1 Security starts with assets and trust

Before reading individual functions, identify assets, privileged actors, upgrade keys, external dependencies, price inputs, and critical invariants. Ask how value can leave the system and who can trigger those paths. Many severe findings are business-logic failures rather than obscure opcodes. Build an attack tree: steal funds, mint unauthorized value, block withdrawals, manipulate price, take governance control, or corrupt accounting.

7.2 Reentrancy

Reentrancy occurs when a contract makes an external call before completing its own state transition and the callee calls back into vulnerable functionality. Classic withdrawal bugs send ETH before reducing balance, enabling repeated withdrawals in one transaction. Cross-function and cross-contract reentrancy can be subtler. Defenses include checks-effects-interactions, reentrancy guards, pull-payment patterns, and minimizing external calls. Do not assume a particular recipient is "just a wallet"; contracts can receive calls.

7.3 Access-control bugs

Common failures include missing modifiers, using `tx.origin``, insecure initialization, role-admin mistakes, unauthorized upgrade paths, trusting an arbitrary caller-provided address, and exposing generic execution functions. Review every privileged function and trace who can become privileged. Verify role transfer/revocation

semantics. In multi-contract systems, an authorized router may accidentally let users choose dangerous call targets, creating a confused-deputy vulnerability.

7.4 Denial of service and call failures

Loops over unbounded user-controlled arrays can make functions uncallable as state grows. Pushing payments to many recipients can fail if one recipient reverts. External protocol calls can pause or revert unexpectedly. Design systems so one bad recipient or dependency does not permanently trap everyone's funds. Use pull-based claims, pagination/batching, emergency paths, and dependency isolation where appropriate.

7.5 Transaction ordering and MEV

Public mempools expose pending intent. Attackers can reorder around swaps, liquidations, NFT mints, auctions, and oracle-sensitive actions. Slippage bounds, deadlines, commit-reveal, private transaction channels, batch auctions, or protocol-specific designs can mitigate different risks. Front-running is not automatically a bug; the question is whether transaction ordering violates user or protocol security assumptions.

Code / Architecture Example

```
// Vulnerable pattern
function withdraw() external {
    uint256 amount = balances[msg.sender];
    (bool ok,) = msg.sender.call{value: amount}(""); // external control first
    require(ok);
    balances[msg.sender] = 0; // effect too late
}

// Safer pattern: validate -> update accounting -> external interaction
```

Hands-On Lab (45-55 minutes)

24. Build `VulnerableBank.sol` and an attacker contract on Anvil.
25. Demonstrate reentrant withdrawals in a test.
26. Patch the bank using checks-effects-interactions and optionally ReentrancyGuard.
27. Write a one-page finding with impact, proof of concept, and remediation.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Can `nonReentrant` fix every reentrancy issue?

A: No. It protects guarded call paths but does not replace correct state design; cross-contract or unguarded functions may remain vulnerable.

Q: Why is tx.origin dangerous for authorization?

A: An attacker contract can trick an EOA into initiating a call chain where tx.origin is the victim while msg.sender is the attacker contract.

Q: What is sandwiching?

A: Ordering a buy before and a sell after a victim swap to profit from the victim's price impact, typically when slippage permits it.

Homework

Review five functions from a known protocol and mark every external call and privileged path.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 8 - Smart Contract Security II - Audit Methodology, Static Analysis, Fuzzing, Signatures, Oracles, and Upgrade Risk

Lecture goal: Perform a structured mini-audit and write findings in professional form.

Portfolio output: Mini-audit PDF/Markdown report for portfolio.

Learning Outcomes

- Create an audit scope, trust model, and threat model.
- Use Slither/static analysis as an assistant rather than an oracle.
- Review signatures, replay protection, oracle assumptions, and proxy storage.
- Write severity-ranked findings with reproducible proof of concept.

60-Minute Teaching Plan

Time	Focus
0-5 min	Audit workflow
5-15 min	Threat modeling
15-25 min	Static/dynamic tooling
25-35 min	Signature/oracle risks
35-45 min	Proxy/storage risks
45-55 min	Mini-audit lab
55-60 min	Report review

8.1 Audit workflow

A strong audit begins with scope and architecture. Read documentation, map contracts and roles, identify external protocol dependencies, list privileged operations, and write protocol invariants. Then perform manual review, targeted tests, fuzz/invariant testing, static analysis, and adversarial scenario simulation. Tools accelerate review but do not understand product intent. Track assumptions and unresolved questions explicitly.

8.2 Severity and exploitability

Severity combines impact and likelihood/exploitability. A critical issue might allow arbitrary theft or protocol takeover with realistic preconditions. High issues cause major loss under plausible conditions. Medium issues damage functionality or funds in narrower scenarios. Low/informational items concern hardening, clarity, or low-impact edge cases. Avoid inflating severity; evidence and realistic attack paths matter.

8.3 Signatures and replay

Signature-based authorization must bind the action to a signer, payload, nonce, deadline when needed, domain/chain/contract context, and intended executor semantics. Omitting nonces allows replay. Weak domain

separation can let a signature be reused across contracts or chains. Verify ECDSA malleability protections through established libraries and handle ERC-1271 contract signatures if smart-account compatibility is required.

8.4 Oracle and economic security

Protocols that price collateral or swaps can be exploited even when Solidity is memory-safe. Review data source, update cadence, stale-price handling, decimals, deviation thresholds, and whether an attacker can manipulate the observed market within one transaction. Thin-liquidity spot prices are dangerous for lending. Flash loans increase available temporary capital but do not create vulnerabilities by themselves; they amplify weak economic assumptions.

8.5 Upgradeability security

Upgradeable contracts add governance and storage risks. Confirm initializers cannot be re-run, implementation contracts are appropriately initialized/locked, upgrade authorization is correct, storage layouts remain compatible, and admin keys are protected by multisig/timelock where appropriate. An upgrade key is effectively a key to the protocol unless governance constrains it.

Code / Architecture Example

Audit finding template

```
Title: Unauthorized mint through misconfigured role admin
Severity: High
Affected: Token.sol::grantRole / role hierarchy
Impact: Attacker can gain MINTER_ROLE and inflate supply
Root cause: Publicly reachable admin path
Proof of concept: Foundry test demonstrating unauthorized role acquisition
Recommendation: Restrict role admin and transfer admin to governed multisig
```

Hands-On Lab (45-55 minutes)

28. Run Slither against your Day 5 token and classify warnings as true positive, false positive, or design note.
29. Perform manual review using an asset/role/external-call checklist.
30. Add one signature replay test or oracle-staleness test to a sample contract.
31. Write a 2-3 page mini audit report.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why can an oracle cause a protocol exploit without a Solidity bug?

A: The code may correctly trust a price source whose economic assumptions are manipulable or stale.

Q: What is domain separation?

A: Binding signed data to a context such as chain, verifying contract, name, and version to prevent cross-domain replay.

Q: Why should an implementation contract be initialized/locked?

A: To prevent someone from taking ownership or privileged state on the implementation itself when that could affect upgrade or operational assumptions.

Homework

Produce a threat model for your capstone idea with assets, actors, trust assumptions, entry points, and ten attack scenarios.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 9 - DeFi Foundations - AMMs, Liquidity, Slippage, LP Accounting, and DEX Integration

Lecture goal: Understand DeFi math and integration risk rather than treating routers as black boxes.

Portfolio output: DEX fork-integration notebook/test with price-impact analysis.

Learning Outcomes

- Explain constant-product AMMs and price impact mathematically.
- Distinguish slippage tolerance from price impact.
- Explain LP shares, fees, impermanent loss, and concentrated liquidity conceptually.
- Integrate a router safely in a fork test.

60-Minute Teaching Plan

Time	Focus
0-5 min	DeFi mental model
5-15 min	Constant-product AMMs
15-25 min	LP accounting and fees
25-35 min	Slippage/price impact/MEV
35-45 min	Router integration
45-55 min	Fork lab
55-60 min	Review

9.1 AMM pricing

A constant-product pool maintains an invariant approximately expressed as $x*y=k$ before fees and implementation details. A swap adds one asset and removes another, moving the reserve ratio and therefore the marginal price. Larger trades relative to liquidity cause larger price impact. This is a mechanism, not an oracle guarantee: a pool's instantaneous price can be manipulated if liquidity is limited.

9.2 Liquidity providers

LPs supply assets and receive claims on pool reserves. Fees compensate LPs for providing liquidity, while changing relative prices can create impermanent loss compared with simply holding the assets. LP token/share accounting must handle deposits, withdrawals, fees, rounding, and first-liquidity edge cases. Concentrated-liquidity AMMs let LPs allocate liquidity to price ranges, improving capital efficiency but making position management more complex.

9.3 Slippage and price impact

Price impact is the market movement caused by the trade itself; slippage is the difference between expected and actual execution, affected by price movement, ordering, and liquidity changes. A router call with `amountOutMin=0``

effectively accepts arbitrary output and is usually unsafe. Deadlines and minimum-output bounds protect users from stale or reordered transactions but must be calculated from reliable quotes.

9.4 Router integration

Before approving a router, validate the target address for the intended chain and protocol version. Handle token decimals and approve only what is needed where practical. Nonstandard tokens may have transfer fees, unusual approval behavior, or return-value quirks. Use fork tests against real contracts and simulate failure modes such as expired deadline, insufficient output, paused token, or changed allowance.

9.5 DEX architecture

Separate pool/pair contracts, factories, routers, quoters, and periphery libraries. The pool owns liquidity and pricing state; the router typically orchestrates transfers and calls. Integrations should not assume a router quote is guaranteed execution. For advanced systems, obtain quotes off-chain, apply safety margins, simulate the final transaction, and verify post-trade balances.

Code / Architecture Example

Constant product example

```
Reserve X = 100 ETH
Reserve Y = 300,000 USDC
k = 30,000,000
```

A trade that adds ETH removes USDC and changes Y/X .
The execution price differs from the pre-trade spot price because reserves move.

Safety rule: never use `amountOutMin = 0` in production swaps unless arbitrary output is truly acceptable.

Hands-On Lab (45-55 minutes)

32. Fork an EVM chain at a fixed block.
33. Read reserves/liquidity from a known pool.
34. Quote a small and a much larger trade and compare price impact.
35. Execute a simulated swap with a realistic `amountOutMin` and verify balances after the swap.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Slippage vs price impact?

A: Price impact is caused by your trade moving the pool; slippage is broader execution deviation from expectation.

Q: Why is a DEX spot price unsafe as a lending oracle?

A: An attacker may move the pool price temporarily if liquidity is insufficient relative to available capital.

Q: What does concentrated liquidity change?

A: LP capital is allocated to selected price ranges, increasing capital efficiency but creating range-dependent exposure and more complex positions.

Homework

Write pseudocode for a safe swap function including approvals, slippage, deadline, recipient validation, and balance checks.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 10 - Lending, Liquidations, Flash Loans, Arbitrage, and MEV-Aware Execution

Lecture goal: Understand lending economics and design a realistic arbitrage simulator without confusing temporary capital with guaranteed profit.

Portfolio output: DeFi arbitrage simulator with risk documentation.

Learning Outcomes

- Explain collateral, LTV, liquidation threshold, health factor, utilization, and interest-rate models.
- Describe flash-loan atomicity and repayment guarantees.
- Calculate arbitrage profit after fees, slippage, gas, and price impact.
- Understand mempool competition and failed-trade risk.

60-Minute Teaching Plan

Time	Focus
0-5 min	Lending overview
5-15 min	Collateral and liquidation
15-25 min	Interest and utilization
25-35 min	Flash loans
35-45 min	Arbitrage/MEV economics
45-55 min	Simulation lab
55-60 min	Review

10.1 Lending protocol accounting

Users supply assets to earn yield and borrow against collateral. Protocols define maximum loan-to-value, liquidation thresholds, reserve factors, caps, and collateral eligibility. A health metric determines whether a position can be liquidated. Senior engineers focus on oracle correctness, decimals, interest accrual, share/index accounting, rounding, and liquidation incentives because tiny accounting errors can become systemic at scale.

10.2 Liquidations

When collateral value falls relative to debt, liquidators repay some debt and receive collateral at a discount/bonus. This creates an incentive to restore solvency. Edge cases include stale prices, large positions relative to market liquidity, bad debt, chain congestion, and correlated collateral. A protocol can be logically correct yet insolvent if liquidation assumptions fail under market stress.

10.3 Flash loans

A flash loan lends assets without traditional collateral provided the entire operation repays principal plus fee before the transaction ends. If repayment fails, the transaction reverts atomically. Flash loans are useful for arbitrage,

collateral swaps, refinancing, liquidations, and migration. They do not guarantee profit and do not bypass protocol authorization; they simply provide temporary capital inside one atomic transaction.

10.4 Arbitrage economics

A profitable-looking price difference can disappear after pool fees, flash-loan premium, gas, price impact, and competing transactions. Bots should simulate exact routes on current state, compute conservative minimum outputs, enforce a minimum profit on-chain, and avoid submitting if expected edge is too small. Searchers compete on latency, routing, private order flow, and bidding for inclusion. Most tutorial bots are not economically competitive.

10.5 MEV-aware execution

Pending transactions can change state before yours executes. Use call simulation on the exact transaction, realistic gas estimation, max-fee strategy, deadlines, and revert-on-unprofitability guards. Private relays/order flow can reduce exposure but introduce provider trust and availability tradeoffs. Build systems that fail safely: a reverted transaction should lose only gas, not principal.

Code / Architecture Example

```
profit = finalAssetBalance
        - borrowedPrincipal
        - flashLoanFee
        - gasCostInAssetTerms

require(profit >= minProfit, "not profitable");

Execution path:
Flash loan -> DEX A swap -> DEX B swap -> repay -> retain residual profit
```

Hands-On Lab (45-55 minutes)

36. Build a local/fork arbitrage simulator between two mock or forked pools.
37. Calculate gross spread and net profit after fees.
38. Add `minProfit` and minimum-output checks.
39. Write tests for profitable, unprofitable, and adverse-price-movement scenarios.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why can a flash loan amplify an exploit?

A: It gives an attacker large temporary capital, which can magnify protocols that rely on manipulable price or liquidity assumptions.

Q: What should an arbitrage contract verify on-chain?

A: At minimum repayment ability and required minimum profit; route-specific slippage/deadline checks should also protect execution.

Q: Why do profitable simulations still fail live?

A: State changes, competing searchers, gas bidding, latency, reordering, and liquidity movement between simulation and inclusion.

Homework

Create a spreadsheet or script that calculates net arbitrage profit for configurable fees, gas, and price impact.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 11 - Oracles, Price Feeds, VRF, Automation, and Data Validation

Lecture goal: Use external data safely by understanding freshness, trust, decimals, failure modes, and fallback design.

Portfolio output: Validated oracle consumer and oracle-security checklist.

Learning Outcomes

- Explain the oracle problem and decentralized feed architecture.
- Consume a price feed while validating decimals and freshness.
- Understand VRF and automation use cases.
- Design an oracle policy for a lending or vault protocol.

60-Minute Teaching Plan

Time	Focus
0-5 min	Oracle problem
5-15 min	Price feed architecture
15-25 min	Freshness and validation
25-35 min	VRF/automation
35-45 min	Oracle attacks and fallback
45-55 min	PriceConsumer lab
55-60 min	Review

11.1 The oracle problem

Smart contracts execute deterministically from chain state and cannot directly fetch arbitrary web data during execution. Oracles bridge external information such as asset prices, weather, sports results, or randomness into blockchain-readable state. The security of the consuming protocol can become no stronger than the oracle assumptions it relies on.

11.2 Price feeds

A price feed commonly aggregates observations from multiple sources/nodes and exposes the latest round answer plus metadata. Integrators must handle signedness, decimals, update timestamps, round completeness, and chain-specific feed addresses. Never assume all feeds use 18 decimals. Store configuration explicitly and test against real feed interfaces on a fork.

11.3 Freshness and circuit breakers

A price can be technically valid yet too old for a fast market. Define a maximum age appropriate to the asset and protocol. Consider deviation checks, sequencer uptime on L2s, fallback logic, and emergency pause behavior.

Fallback oracles can improve availability but may introduce inconsistent pricing or a weaker trust model; document the hierarchy clearly.

11.4 Randomness and automation

Verifiable randomness frameworks provide proofs that randomness was generated according to protocol rules, useful for lotteries, gaming, NFT allocation, and sampling. Automation services trigger functions based on time or conditions, but contracts must still validate that the action is safe when executed. Never encode an assumption that an off-chain keeper will always run on schedule.

11.5 Oracle attack review

Review whether an attacker can influence the source market, delay updates, exploit stale data, exploit decimal mistakes, or cause a fallback to weaker pricing. For collateralized protocols, perform market-depth and stress analysis in addition to code review. Oracle configuration is governance-sensitive and should be monitored continuously.

Code / Architecture Example

```
function readPrice() external view returns (uint256) {
    (, int256 answer,, uint256 updatedAt,) = feed.latestRoundData();
    require(answer > 0, "bad price");
    require(block.timestamp - updatedAt <= MAX_AGE, "stale price");
    return uint256(answer);
}

// Also inspect feed.decimals() and normalize carefully.
```

Hands-On Lab (45-55 minutes)

40. Create a PriceConsumer using a test/fork feed.
41. Reject zero/negative prices and stale timestamps.
42. Normalize the feed value to an application-specific decimal format.
43. Write tests for valid, stale, and invalid data using mocks.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why does freshness matter?

A: A stale valid price can cause incorrect collateral, liquidation, or swap decisions during fast markets.

Q: Why not always use a DEX spot price?

A: It may be manipulable within one or a few transactions, especially in thin liquidity.

Q: What is the purpose of VRF?

A: Provide verifiable unpredictable randomness rather than trusting a single operator to choose outcomes.

Homework

Draft an oracle policy for ETH/USD collateral including max age, fallback behavior, pause conditions, and monitoring alerts.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 12 - Upgradeable Contracts - Transparent/UUPS Proxies, Initializers, Storage Layout, and Governance

Lecture goal: Implement and review proxy upgrades without corrupting state or creating a governance backdoor.

Portfolio output: Upgradeable V1/V2 project with storage-layout and governance documentation.

Learning Outcomes

- Explain proxy/implementation separation and delegatecall.
- Compare Transparent and UUPS approaches.
- Use initializers and storage-safe upgrades.
- Design upgrade authorization with multisig/timelock controls.

60-Minute Teaching Plan

Time	Focus
0-5 min	Why upgradeability exists
5-15 min	Proxy architecture
15-25 min	Initializer pattern
25-35 min	Storage compatibility
35-45 min	Transparent vs UUPS/governance
45-55 min	V1->V2 lab
55-60 min	Review

12.1 Upgradeability tradeoff

Immutable contracts minimize governance power but cannot fix bugs or add features. Proxy-based upgradeability keeps a stable user-facing proxy address while swapping the logic implementation. This creates powerful operational flexibility and a powerful attack surface. Treat the upgrade authority as equivalent to control over all assets governed by the proxy unless technical constraints limit it.

12.2 Proxy mechanics

Users call the proxy. The proxy delegates execution to implementation code using `delegatecall`, so the proxy storage holds protocol state while implementation bytecode supplies logic. Special EIP-1967-style storage slots separate implementation/admin metadata from ordinary application storage. Tooling abstracts much of this, but reviewers must still understand the underlying context.

12.3 Initializers

Constructors run only when implementation bytecode is deployed, not each time code is executed through a proxy. Upgradeable implementations therefore use initializer functions guarded so they can run only at the appropriate version. Inheritance requires careful initializer chaining. Uninitialized proxies or implementations can lead to ownership takeover depending on architecture.

12.4 Storage compatibility

Upgrades must preserve the meaning and location of existing storage. Reordering variables, changing types, or modifying incompatible inherited storage can corrupt state. Append new variables according to the framework's storage-layout rules and use automated upgrade validation. When changing dependency major versions, verify storage compatibility rather than assuming semantic-version API compatibility implies safe layout migration.

12.5 Governance and rollout

Production upgrades should use controlled deployment scripts, bytecode verification, storage-layout comparison, simulation/fork tests, multisig authorization, timelocks when appropriate, and monitoring after execution. Emergency upgrade paths may need shorter delays but increase governance risk. Document who can upgrade, under what conditions, and how users can observe pending changes.

Code / Architecture Example

```
// Conceptual UUPS-style implementation
contract VaultV2 is Initializable, UUPSUpgradeable, AccessControlUpgradeable {
    function initialize(address admin) public initializer {
        __AccessControl_init();
        __UUPSUpgradeable_init();
        __grantRole(DEFAULT_ADMIN_ROLE, admin);
    }

    function _authorizeUpgrade(address) internal override onlyRole(DEFAULT_ADMIN_ROLE) {}
}
```

Hands-On Lab (45-55 minutes)

44. Deploy V1 proxy locally using upgrade tooling.
45. Set state in V1, upgrade to V2, and prove state remains unchanged.
46. Add a new V2 variable/function without modifying old variable order.
47. Write tests that unauthorized users cannot upgrade.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Where is state stored in a proxy system?

A: Primarily in the proxy storage; implementation code interprets that storage through delegatecall.

Q: Why are constructors replaced by initializers?

A: Constructor execution occurs on implementation deployment and does not initialize proxy storage.

Q: What is the biggest governance risk?

A: A compromised or abusive upgrade authority can deploy malicious logic that changes protocol behavior or drains assets.

Homework

Write a production upgrade runbook: prechecks, simulation, approvals, execution, verification, monitoring, and rollback/emergency actions.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 13 - Web3 Backend Engineering - TypeScript/Python, RPC Reliability, Signing, Nonces, and APIs

Lecture goal: Connect contracts to production services with safe key handling, resilient RPC usage, and deterministic transaction management.

Portfolio output: Web3 backend service with safe architecture notes.

Learning Outcomes

- Use ethers/viem or web3.py to read/write contracts.
- Separate read-only RPC operations from signing infrastructure.
- Manage nonces, retries, receipts, and replacement transactions.
- Design a backend API without exposing private keys.

60-Minute Teaching Plan

Time	Focus
0-5 min	Backend architecture
5-15 min	Reads and ABI interaction
15-25 min	Transaction building/signing
25-35 min	Nonce/fee/retry management
35-45 min	Key security and provider resilience
45-55 min	REST API lab
55-60 min	Review

13.1 Backend responsibilities

A blockchain backend often coordinates off-chain business rules, database state, RPC reads, transaction submission, event indexing, and user notifications. Do not make the backend a hidden custodian unless custody is a deliberate product requirement. User-owned actions should usually be signed in the user wallet; server-owned operational actions should use dedicated service keys or custody infrastructure with least privilege.

13.2 Reads and calls

Read operations such as balances and view-function calls can be served through one or more RPC providers and cached where appropriate. Specify block tags when consistency matters. A read at `latest` followed by another read may observe a different block, so critical workflows can pin block numbers. Batch or multicall reads to reduce latency when supported.

13.3 Transaction manager

A production transaction manager tracks account nonce, estimated gas, fee policy, submitted transaction hashes, receipts, replacements, and failures. Concurrent workers using one signer can collide on nonces without coordination. Persist pending transaction state so a process restart does not forget what has already been broadcast. Distinguish deterministic contract reverts from network/provider failures.

13.4 Key security

Never store raw private keys in frontend code, source control, logs, or analytics. Backend keys should live in a secret manager, HSM, KMS, or managed custody system appropriate to the risk level. Use dedicated keys with minimal balances/permissions. High-value governance should use multisigs rather than unattended hot keys.

13.5 RPC resilience

Providers can rate-limit, return transient errors, fall behind, or disagree about pending state. Implement retry policies for idempotent reads, cautious retry for writes, endpoint health checks, and fallback providers. Never blindly rebroadcast a newly signed transaction if you are uncertain whether the first submission succeeded; first check transaction/nonce state.

Code / Architecture Example

```
// TypeScript / viem-style concept
const balance = await publicClient.readContract({
  address: token,
  abi,
  functionName: 'balanceOf',
  args: [user],
});

// Server signing should use secure key management, not a key embedded in source.
```

Hands-On Lab (45-55 minutes)

48. Build endpoints: GET native balance, GET ERC-20 balance, GET transaction receipt.
49. Add one server-owned test transaction endpoint restricted to local/testnet usage.
50. Persist submitted tx hash and status in SQLite/Postgres.
51. Simulate RPC failure and demonstrate failover/read retry behavior.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why are nonces difficult in concurrent backends?

A: Multiple workers can attempt to use the same next nonce unless submissions are serialized/coordinated.

Q: Should a backend sign every user transaction?

A: Usually no; that makes the backend custodial. User-wallet signing preserves user control unless custody is a stated requirement.

Q: Why pin a block for related reads?

A: To get a consistent snapshot rather than mixing values from different chain states.

Homework

Design a transaction-status state machine: created, signed, submitted, pending, replaced, mined-success, mined-revert, dropped, finalized.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 14 - Indexing, Events, The Graph, Databases, Reorg Handling, and Analytics

Lecture goal: Build reliable off-chain views of blockchain history and understand why logs must be indexed with reorg awareness.

Portfolio output: Event indexer and analytics API.

Learning Outcomes

- Decode event topics/data and design query-friendly events.
- Build an event indexer into a relational database.
- Explain reorg handling, checkpoints, and idempotency.
- Understand subgraph-style indexing and GraphQL APIs.

60-Minute Teaching Plan

Time	Focus
0-5 min	Why indexing exists
5-15 min	Logs/topics/events
15-25 min	Indexer architecture
25-35 min	Reorg/idempotency
35-45 min	Subgraphs/GraphQL
45-55 min	Dashboard lab
55-60 min	Review

14.1 Events as off-chain data

Events are cheaper than storing query-oriented history in contract storage and are the standard interface for off-chain consumers. Indexed parameters become topics, which can be filtered efficiently. Design events so a downstream system can reconstruct important state transitions without relying on opaque traces. Events are not an authorization mechanism and not readable by contracts after the transaction.

14.2 Indexer pipeline

A basic indexer fetches block ranges or subscribes to logs, decodes events using ABI, writes normalized rows, and tracks a checkpoint. It should be idempotent so reprocessing the same block/log does not duplicate data. Use unique keys such as chain ID + transaction hash + log index. Persist block hash alongside block number to detect reorgs.

14.3 Reorg handling

If a previously indexed block is replaced, associated events may disappear or change. An indexer can wait for confirmations before marking records final, and it should be able to roll back records from orphaned blocks. The

required confirmation depth depends on chain/finality model and business risk. Deposits to exchanges or bridges often use stronger finality thresholds than a UI activity feed.

14.4 Subgraphs and GraphQL

Subgraph-style systems define data sources, event handlers, entity schemas, and GraphQL queries so applications can retrieve structured blockchain data. This reduces custom infrastructure for many dApps. However, teams still need to understand indexing latency, reorg behavior, entity design, and hosted/decentralized service trust assumptions.

14.5 Analytics design

Separate raw immutable events from derived business tables. Raw data enables reprocessing when decoding logic changes. Derived views can aggregate TVL, volume, fees, active users, positions, or PnL. Store token decimals and price sources explicitly; analytics errors often come from unit normalization rather than blockchain retrieval.

Code / Architecture Example

```
Indexer uniqueness key:
(chain_id, tx_hash, log_index)

Checkpoint table:
chain_id | last_block_number | last_block_hash | finalized_block

Reorg rule:
if stored hash(block N) != canonical hash(block N):
    rollback affected derived/raw records and replay from safe ancestor
```

Hands-On Lab (45-55 minutes)

52. Index Transfer/Deposit/Withdrawal events into SQLite/Postgres.
53. Expose `/events` and `/balances-derived` endpoints.
54. Store block hash and log index.
55. Simulate reprocessing the same block and prove no duplicates are inserted.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Why store logIndex?

A: One transaction can emit multiple events; tx hash alone is not unique per log.

Q: How do you handle reorgs?

A: Track block hashes/checkpoints, detect mismatches, rollback orphaned data, replay from a safe ancestor, and distinguish confirmed/finalized data.

Q: Why keep raw events?

A: They allow deterministic reprocessing when business logic, schemas, or aggregations change.

Homework

Design database tables for a lending protocol: markets, supplies, borrows, liquidations, prices, blocks, raw_logs.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 15 - Layer 2 and Rollups - Optimistic, ZK, Sequencers, Bridges, Data Availability, and Finality

Lecture goal: Explain modern scaling architectures and the operational differences dApps face on L2s.

Portfolio output: L2 deployment report and architecture comparison.

Learning Outcomes

- Differentiate L1, sidechains, optimistic rollups, and validity/ZK rollups.
- Explain sequencing, batching, settlement, proofs, and data availability.
- Understand L1/L2 message delays and withdrawal semantics.
- Deploy and compare an app across EVM L1/L2 test environments.

60-Minute Teaching Plan

Time	Focus
0-5 min	Scaling problem
5-15 min	Rollup architecture
15-25 min	Optimistic rollups
25-35 min	Validity/ZK systems
35-45 min	Bridges/finality/operations
45-55 min	Deployment lab
55-60 min	Review

15.1 Scaling layers

Layer 1 provides base consensus and settlement. Layer 2 systems execute transactions away from L1 while publishing commitments and/or data that allow the system to derive security from L1 according to the rollup design. Sidechains have their own consensus and therefore a different trust model even if they are EVM-compatible.

15.2 Rollup components

Typical components include an L2 execution environment, sequencer, batch poster, bridge/inbox/outbox contracts, proof or dispute system, and data-availability mechanism. Users experience faster/cheaper execution but must understand how and when L2 state becomes anchored to L1. Sequencer downtime can affect transaction inclusion even when funds remain recoverable through protocol-specific escape mechanisms.

15.3 Optimistic rollups

Optimistic systems generally assume posted state transitions are valid unless challenged during a dispute window. Fraud-proof/dispute mechanisms and delayed finality influence canonical withdrawals and bridge UX. Application-level "confirmed" on L2 is not necessarily the same as economically finalized on L1.

15.4 Validity/ZK rollups

Validity systems post cryptographic proofs that state transitions satisfy the rollup circuit/protocol rules. This can provide faster L1 assurance once proofs are verified, but proving systems, data availability, and bridge design remain complex. The term "ZK" is often used broadly; understand whether privacy is actually part of a particular system.

15.5 Application differences

Even EVM-equivalent L2s can have chain-specific gas accounting, precompiles, message-passing contracts, sequencer feeds, fee components, and finality behavior. Oracle consumers may need sequencer-uptime checks. Cross-domain messages can be asynchronous. Always read target-chain deployment and bridging documentation rather than assuming "same EVM" means identical operations.

Code / Architecture Example

```
L2 mental model

User tx -> L2 sequencer -> L2 execution
        -> batch/state commitment -> L1
        -> proof or dispute process -> stronger settlement/finality

Cross-domain message:
L1 contract -> bridge/inbox -> L2 message execution (asynchronous)
```

Hands-On Lab (45-55 minutes)

56. Deploy the same simple contract to two EVM test environments.
57. Record chain ID, RPC URL type, explorer, gas estimate, transaction finality behavior, and native gas token.
58. Document canonical bridge deposit/withdrawal flow conceptually.
59. Identify any chain-specific oracle/sequencer considerations.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Rollup vs sidechain?

A: A rollup derives security/settlement from an L1 protocol through posted data/commitments and proof/dispute rules; a sidechain primarily relies on its own consensus.

Q: Why can L2 withdrawals take longer?

A: Some designs require proof/dispute/finality periods before an L1 withdrawal is considered canonical.

Q: What is a sequencer?

A: A service/component that orders and typically executes/accepts L2 transactions before batches are committed to L1.

Homework

Create a comparison matrix for Ethereum mainnet, one optimistic rollup, and one validity rollup: fees, finality, bridge model, sequencer assumptions, EVM compatibility.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 16 - Cross-Chain Systems and Bridge Security - Messaging, Token Transfer, Validation, Replay, and Rate Limits

Lecture goal: Design cross-chain interactions with explicit trust and failure assumptions.

Portfolio output: Cross-chain architecture and threat-model document.

Learning Outcomes

- Explain lock/mint, burn/mint, and liquidity-network transfer models.
- Separate cross-chain messaging from token accounting.
- Identify replay, forged-message, validator, and rate-limit risks.
- Design a bridge-aware token transfer workflow.

60-Minute Teaching Plan

Time	Focus
0-5 min	Cross-chain problem
5-15 min	Bridge models
15-25 min	Messaging lifecycle
25-35 min	Security failures
35-45 min	Rate limits/recovery
45-55 min	Architecture lab
55-60 min	Review

16.1 Bridge models

A bridge moves value or messages between chains that do not natively share one execution state. Lock/mint systems lock canonical assets on a source chain and mint representations on a destination. Burn/mint systems can burn on one chain and mint on another when token governance spans chains. Liquidity networks pay users from destination liquidity and settle between operators later. Each model shifts risk among custody, messaging, liquidity, and governance.

16.2 Message lifecycle

A source-chain contract emits or stores a message. An off-chain/consensus network observes finality, validates the message under its rules, and submits proof/attestation to a destination-chain contract. The destination verifies origin, source sender, nonce/message ID, payload, and authorization before executing. Application contracts should validate expected remote counterpart addresses rather than trusting any message that the transport layer accepts.

16.3 Bridge security

Historical bridge failures often involve compromised validator keys, weak multisig thresholds, forged proofs, replay, incorrect message verification, upgrade compromise, or accounting mismatches between locked and minted supply. Cross-chain code compounds security because bugs span two chains plus off-chain relayers/validators. Treat bridge dependencies as critical external protocols.

16.4 Replay and idempotency

Every message should have a unique identifier or nonce and destination contracts should reject already-processed messages. Idempotent design reduces damage if a relay submits the same valid message multiple times. Include chain/domain identifiers and expected sender in signed/verified payloads to prevent cross-domain confusion.

16.5 Rate limits and recovery

Large bridges should consider per-asset caps, rate limits, delayed high-value transfers, pausing, monitoring, and incident procedures. Rate limits do not fix correctness bugs, but they can limit blast radius. Design failure handling for a message that is finalized on source but temporarily unexecutable on destination.

Code / Architecture Example

Cross-chain validation checklist

- expected source chain/domain
- expected source application address
- unique nonce/message ID
- destination chain/application binding
- payload hash and token/amount validation
- replay protection
- rate limit / cap
- pause/emergency control
- observability event

Hands-On Lab (45-55 minutes)

60. Draw a token bridge with source escrow, messenger, verifier, and destination wrapped token.
61. Specify exactly who can mint on destination and what proof authorizes it.
62. Add replay-protection pseudocode and per-day transfer cap.
63. Write three incident scenarios and recovery actions.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: What is the main bridge security question?

A: Who/what is trusted to attest that a source-chain event really happened and is final, and can that authority be forged or compromised?

Q: Why check remote sender in the application?

A: A generic messaging network may deliver messages from many applications; the destination must bind messages to the intended counterpart.

Q: Why use rate limits?

A: They can cap the value lost during key compromise, verifier bug, or unexpected incident while operators respond.

Homework

Threat-model a bridge with validator quorum, upgrade admin, source escrow, destination mint authority, and monitoring.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 17 - Solana, Rust, Anchor, Accounts, PDAs, CPIs, SPL Tokens, and Program Deployment

Lecture goal: Become capable of understanding and building non-EVM programs using Solana's account model.

Portfolio output: Solana Anchor counter program with tests and EVM/Solana comparison.

Learning Outcomes

- Explain Solana accounts, programs, instructions, transactions, signers, and account ownership.
- Use Rust/Anchor account validation.
- Derive and use PDAs conceptually.
- Understand CPI and SPL token account flows.

60-Minute Teaching Plan

Time	Focus
0-5 min	EVM vs Solana
5-15 min	Accounts/programs/instructions
15-25 min	Rust/Anchor structure
25-35 min	PDAs and account constraints
35-45 min	CPI/SPL tokens
45-55 min	Counter lab
55-60 min	Review

17.1 Solana execution model

Solana programs are executable code while mutable application state lives in accounts passed to instructions. Programs do not implicitly own a single storage namespace like EVM contracts. Each account has metadata including owner program, lamports, and data. Transactions specify instructions and required accounts, allowing the runtime to know read/write dependencies and execute compatible transactions in parallel.

17.2 Anchor structure

Anchor uses Rust macros to reduce boilerplate, define instruction handlers, validate accounts, serialize data, derive clients/IDLs, and improve testing/deployment workflows. `#[derive(Accounts)]` captures constraints such as signer requirements, ownership, seeds, initialization, and mutability. Security review focuses heavily on whether the correct accounts are validated.

17.3 Program Derived Addresses

PDAs are deterministic addresses derived from seeds plus a program ID and are designed so no ordinary private key controls them. Programs can "sign" for their PDAs through runtime rules. PDAs are used for vaults, configuration accounts, user state, escrow authorities, and deterministic namespaces. Seed design and bump handling should avoid collisions and confusion.

17.4 Cross-Program Invocation

A CPI lets one Solana program invoke another. The calling program passes accounts and instruction data, and signer privileges can propagate under rules including PDA signing. This enables composition with the System Program, SPL Token Program, Associated Token Program, DEXs, and lending protocols. Validate downstream program IDs and all involved accounts; account substitution is a common Solana security theme.

17.5 SPL tokens

SPL token state is separated among mint accounts and token accounts/associated token accounts. A mint defines supply/decimals and authorities; token accounts hold balances for an owner. Closing an eligible token account can reclaim rent/lamports to a destination under the token program's authority rules. Program executable accounts and program-data accounts have separate upgrade/deployment semantics from ordinary token accounts.

Code / Architecture Example

```
#[program]
pub mod counter {
    use super::*;
    pub fn increment(ctx: Context<Increment>) -> Result<()> {
        ctx.accounts.counter.value += 1;
        Ok(())
    }
}

#[derive(Accounts)]
pub struct Increment<'info> {
    #[account(mut)]
    pub counter: Account<'info, Counter>,
    pub authority: Signer<'info>,
}

#[account]
pub struct Counter { pub value: u64 }
```

Hands-On Lab (45-55 minutes)

64. Install Rust/Solana/Anchor in a supported local environment (Windows users can use WSL).
65. Create an Anchor counter program with an initialized state account.
66. Add an authority constraint so only the expected signer can increment.
67. Run local validator tests and inspect program/account addresses.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: Who owns state in Solana?

A: State is stored in accounts whose owner field identifies the program allowed to modify their data under runtime rules.

Q: What is a PDA?

A: A deterministic program-derived address with no private key that a program can authorize using seeds/bump through runtime signing rules.

Q: Why are account constraints critical?

A: Programs receive accounts from transaction callers; failing to validate ownership, signer, seeds, or relationships can let attackers substitute malicious accounts.

Homework

Create a comparison document: EVM contract storage vs Solana accounts; msg.sender vs signers; contract calls vs CPI; ERC-20 vs SPL token accounts.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 18 - Senior Blockchain System Architecture - On/Off-Chain Boundaries, Multisig, Timelocks, Monitoring, and Threat Modeling

Lecture goal: Design complete systems, not isolated contracts, and justify every trust boundary.

Portfolio output: Architecture pack: diagrams, role matrix, threat model, and operational checklist.

Learning Outcomes

- Choose on-chain vs off-chain responsibilities based on trust and cost.
- Design admin controls with least privilege, multisig, timelocks, and emergency powers.
- Define monitoring, incident response, and deployment controls.
- Produce an architecture and threat model for a DeFi platform.

60-Minute Teaching Plan

Time	Focus
0-5 min	Architect mindset
5-15 min	On-chain/off-chain boundaries
15-25 min	Governance/privilege
25-35 min	Monitoring/operations
35-45 min	Threat model and failure domains
45-55 min	Architecture workshop
55-60 min	Review

18.1 System decomposition

Start with user journeys and trust requirements, then decide which state must be enforceable on-chain. Keep critical custody and authorization rules on-chain when decentralization is a product goal, while using off-chain services for search, analytics, notifications, simulations, and UX orchestration. Do not put data on-chain merely because it can be; persistent storage is costly and public.

18.2 Privilege design

List every power: mint, pause, upgrade, change oracle, change fees, rescue tokens, set caps, add markets, and move treasury assets. Assign each power to the narrowest role. High-value roles should use multisigs, hardware-backed keys, timelocks, or governance. Emergency pause may intentionally be faster than upgrade authority. Document which powers can directly move user funds.

18.3 Monitoring

Monitor contract events, admin actions, upgrades, oracle freshness/deviation, TVL changes, unusual withdrawals, failed transactions, bridge queues, RPC health, signer balances, and indexer lag. Alerts should have clear owners and runbooks. A protocol without monitoring is difficult to operate safely even if the code is audited.

18.4 Deployment and change management

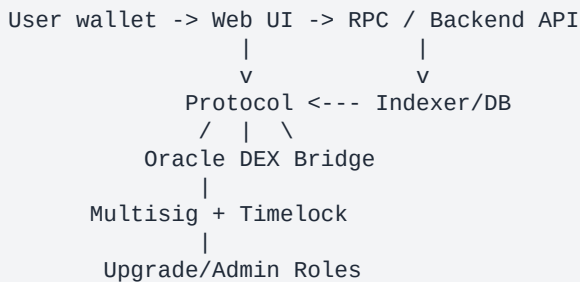
Use reproducible builds, reviewed deployment scripts, chain-specific configuration files, verification, post-deploy checks, and saved transaction artifacts. Production changes should be simulated on forks, reviewed by another engineer, and compared against expected bytecode/storage. Keep an inventory of deployed addresses and admin ownership.

18.5 Threat modeling

Model threats across contracts, backend, frontend, keys, oracles, bridges, dependencies, governance, and users. Consider malicious admins, compromised frontends, poisoned RPC, dependency upgrades, chain halts, sequencer downtime, oracle failure, and extreme market conditions. Architecture is about failure containment as much as happy-path throughput.

Code / Architecture Example

Reference architecture



Monitoring observes every critical edge and privileged action.

Hands-On Lab (45-55 minutes)

68. Design an institutional DeFi vault/lending system.
69. Produce component diagram, trust-boundary diagram, role matrix, and incident runbook outline.
70. For each external dependency, state failure behavior and fallback.
71. Identify at least ten invariants and ten alerts.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: What belongs on-chain?

A: Rules whose integrity must not depend on a trusted server, especially custody/settlement/authorization; noncritical search/analytics/UX can often be off-chain.

Q: Why timelock upgrades?

A: They provide transparency and reaction time before powerful changes execute, reducing surprise-governance risk.

Q: What is a failure domain?

A: A component or trust boundary whose failure can affect a defined subset of the system; architecture should prevent one failure from compromising everything.

Homework

Write a two-page architecture decision record explaining upgradeable vs immutable choice for your capstone.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 19 - Capstone Build - Production DeFi Vault with Tests, Backend, Indexing, and Security Controls

Lecture goal: Integrate the course into one portfolio system that demonstrates senior-level engineering breadth.

Portfolio output: Full DeFi vault capstone repository ready to show clients/interviewers.

Learning Outcomes

- Design vault share accounting and deposit/withdraw flows.
- Implement roles, pause, fees, events, and safe external strategy boundaries.
- Create unit/fuzz/invariant/fork tests.
- Connect backend/indexer and produce deployable documentation.

60-Minute Teaching Plan

Time	Focus
0-5 min	Requirements
5-15 min	Vault/share design
15-25 min	Strategy and privilege boundaries
25-35 min	Testing/security plan
35-45 min	Backend/indexing integration
45-55 min	Build sprint
55-60 min	Acceptance checklist

19.1 Capstone requirements

Build a simplified vault that accepts one ERC-20 asset and issues shares representing proportional ownership. Define deposit, mint, withdraw, redeem, fee behavior, pause rules, and emergency controls before implementation. For learning, keep investment strategy simple; the goal is correct accounting and architecture, not risky yield optimization.

19.2 Share accounting

Vault shares require careful rounding. Deposits convert assets to shares based on total assets and total supply; withdrawals convert shares back to assets. Decide rounding direction intentionally so users cannot exploit repeated tiny deposits/withdrawals to extract value. Handle empty-vault initialization, donations, fees, and strategy-reported asset value. Consider using established vault standards/libraries rather than inventing custom accounting for real production funds.

19.3 Strategy boundary

Separate the vault from external strategy integrations. Limit which strategies are allowed, how much capital can be allocated, and who can change strategy configuration. External protocols can revert, pause, be exploited, or return unexpected token behavior. The vault should have an emergency mode that prioritizes user asset recovery.

19.4 Testing and invariants

Key invariants: total shares cannot be negative/overflow; user cannot redeem more shares than owned; assets should cover redeemable claims under modeled assumptions; pause blocks intended actions; only authorized roles can change strategy/fees; fees stay within caps. Add fuzz sequences of deposits/withdrawals and fork tests for real token behavior if integrating a real asset.

19.5 Production documentation

A strong portfolio repository includes architecture, assumptions, deployment steps, addresses, roles, tests, security considerations, known limitations, incident procedures, and screenshots/API examples. State clearly that a portfolio project is unaudited and not intended for real funds unless it has undergone appropriate review.

Code / Architecture Example

```
defi-vault-capstone/
  contracts/
    Vault.sol
    StrategyAdapter.sol
  test/
    Vault.unit.t.sol
    Vault.fuzz.t.sol
    Vault.invariant.t.sol
    Vault.fork.t.sol
  script/Deploy.s.sol
  backend/
  indexer/
  docs/
    architecture.md
    threat-model.md
    operations.md
  README.md
```

Hands-On Lab (45-55 minutes)

72. Create repository structure and requirements document.
73. Implement minimal vault + tests before external strategy.
74. Add one strategy adapter or mock and failure-path tests.
75. Add backend read endpoints and index deposit/withdraw events.
76. Run complete test suite and generate a final security checklist.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: What is the first vault invariant you check?

A: Asset/share accounting: users should receive/redeem proportional value and no sequence should create value from rounding alone.

Q: Why isolate strategies?

A: External strategy risk should not give arbitrary control over vault logic; adapters and caps reduce blast radius.

Q: What makes this portfolio senior-level?

A: Requirements, architecture, security model, tests, operational thinking, and integration quality - not contract line count.

Homework

Finish the capstone, record a 5-minute demo, and write one mock audit finding against your own design plus remediation.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Day 20 - Senior Blockchain Career Track - Interviews, LinkedIn, Upwork, Proposals, Scoping, and Portfolio

Lecture goal: Convert technical ability into credible job applications and successful freelance delivery.

Portfolio output: Job-ready portfolio checklist, Upwork proposal template, interview question bank, and 90-day plan.

Learning Outcomes

- Position yourself around specific senior blockchain outcomes rather than generic token creation.
- Build a portfolio narrative from the 20-day projects.
- Scope blockchain jobs, identify risk, estimate deliverables, and ask senior-level discovery questions.
- Prepare for Solidity/DeFi/security/system-design interviews.

60-Minute Teaching Plan

Time	Focus
0-5 min	Career positioning
5-15 min	Portfolio and GitHub
15-25 min	LinkedIn positioning
25-35 min	Upwork proposal/scoping
35-45 min	Senior interview topics
45-55 min	System design mock interview
55-60 min	90-day execution plan

20.1 Positioning

Avoid presenting yourself only as a "token developer." Higher-value work includes protocol design, security hardening, upgrade migrations, DeFi integrations, indexing, backend infrastructure, Solana programs, and technical architecture. A credible headline could be "Blockchain / Smart Contract Engineer - Solidity, EVM, DeFi, Foundry, Web3, Solana." Use "senior" when your project history, debugging depth, and ownership justify it; expertise is demonstrated by decisions and outcomes, not a title alone.

20.2 Portfolio narrative

Each repository should answer: What problem does this solve? What are the architecture and trust assumptions? What tests exist? What security considerations were addressed? How is it deployed? What would need to change before production? Pin 4-6 strongest projects rather than dozens of tutorial forks. Include diagrams and concise READMEs that a client can understand in two minutes.

20.3 Scoping client work

A senior freelancer converts vague requests into requirements. For a staking platform ask chain, assets, reward source, reward formula, lockups, early exit, admin powers, upgradeability, emergency controls, expected TVL,

oracle dependencies, frontend/backend scope, audit expectations, deployment environments, and ownership after handoff. Identify third-party contracts and whether the client expects you to custody keys. Put exclusions and acceptance criteria in writing.

20.4 Proposal strategy

Lead with the client's problem and a brief implementation approach. Mention one or two directly relevant examples. Break work into milestones such as architecture, contract implementation, tests, frontend/backend integration, testnet, audit fixes, and mainnet handoff. Do not promise "100% secure" contracts. Instead describe testing, review, threat modeling, and independent audit recommendations for meaningful value.

20.5 Senior interviews

Prepare to explain EVM execution, storage, delegatecall, proxies, reentrancy, access control, EIP-712, AMMs, lending, oracles, flash loans, MEV, fuzz/invariant testing, RPC/indexing, L2s, bridges, Solana PDAs/CPI, and system architecture. Practice reasoning aloud: identify assumptions, failure modes, and tradeoffs. Interviewers often care more about how you reason through an unfamiliar protocol than whether you memorize every API.

Code / Architecture Example

Senior project discovery checklist

1. Target chains and environments?
2. Assets / standards / decimals?
3. Custody and signer model?
4. Roles, upgrades, pause, governance?
5. Oracle / DEX / lending / bridge dependencies?
6. Expected TVL and threat model?
7. Backend/indexer/frontend scope?
8. Testing, audit, monitoring expectations?
9. Deployment and key handoff?
10. Acceptance criteria and maintenance window?

Hands-On Lab (45-55 minutes)

77. Write your LinkedIn headline and About section around blockchain engineering outcomes.
78. Create a one-page Upwork profile with 3 service packages: Smart Contract Development, DeFi Integration, Security/Testing Review.
79. Prepare a proposal for a hypothetical staking-platform job.
80. Run a 10-minute mock interview: design a secure upgradeable DeFi staking platform handling significant TVL.

Instructor Discussion Prompts

- What assumption in today's design would be most dangerous if it were wrong?
- Which part belongs on-chain, and which part is better handled off-chain?
- How would you test the failure path rather than only the happy path?

Senior Interview Questions

Q: How do you answer an unfamiliar protocol question?

A: State assumptions, decompose architecture, identify assets/trust, reason through state transitions, list failure modes, and propose tests rather than bluffing.

Q: What makes a good blockchain proposal?

A: Clear understanding of requirements, concrete milestones, security/testing plan, relevant evidence, realistic exclusions, and production-risk awareness.

Q: What should you never promise?

A: Guaranteed profit, guaranteed security, or production safety without sufficient testing/review and context.

Homework

Create a 90-day plan: 30 days deepen Solidity/security, 30 days build two production-quality portfolio projects, 30 days apply/propose consistently and perform mock interviews.

End-of-Day Acceptance Checklist

- I can explain the main concepts without reading notes.
- I completed or started the lab and can describe the failure modes I tested.
- I committed code/notes to GitHub with a meaningful README.
- I can answer the interview questions in my own words.
- I know what I would harden before using this work with real funds.

Appendix A - Senior Interview Question Bank

EVM: Walk through an external contract call from ABI encoding to opcode execution and returndata.

Storage: How are mappings and dynamic arrays placed in storage, and why does this matter for proxies?

Security: Give an example of a protocol that is code-correct but economically exploitable.

Testing: Design three invariants for a lending protocol.

DeFi: Derive why a constant-product AMM produces price impact.

Oracles: What happens if an oracle is stale but still returns a positive value?

Proxies: Why can a storage-compatible upgrade still be dangerous?

Backend: How would you manage nonces for five workers using one signing account?

Indexing: How would you recover from a 3-block reorg?

L2: What does "finalized" mean differently on L2 versus L1?

Cross-chain: What exactly authorizes minting on the destination chain?

Solana: How can account substitution occur in a Solana program?

Architecture: Design admin roles for a \$50M TVL vault.

Operations: What alerts should fire immediately after an upgrade?

Incident response: The oracle stops updating during market volatility. What does your protocol do?

Mock System Design Prompt

Design a secure upgradeable staking/vault platform handling significant TVL across an EVM L2. Cover contract architecture, accounting, oracle dependencies, roles, upgrade governance, emergency pause, monitoring, indexing, frontend/backend boundaries, tests, deployment, and incident response. State which components can fail without losing user funds.

Appendix B - Portfolio and Upwork Checklist

- Pin 4-6 high-quality GitHub repositories: token/security lab, DeFi integration, upgradeable system, backend/indexer, Solana program, capstone.
- Every repo: problem statement, architecture diagram, setup, tests, deployment, security considerations, limitations, license.
- LinkedIn headline states specialization and stack, not just "Blockchain Developer".
- LinkedIn About section shows outcomes: secure smart contracts, DeFi integrations, testing/auditing, Web3 backend, architecture.
- Upwork proposals begin with the client requirement, propose milestones, and identify security/production risks.
- Never claim guaranteed profit or perfect security; recommend independent audits for high-value contracts.
- Use a discovery checklist before quoting fixed-price blockchain work.
- Keep a reusable proposal template but customize technical approach for every client.
- Prepare 5-minute demos for capstone projects so recruiters can evaluate quickly.
- Continue beyond this 20-day course with 60-90 days of implementation, audits, and open-source contribution.

Recommended 90-Day Continuation

Days 1-30	Days 31-60	Days 61-90
Deepen Solidity, EVM internals, Foundry, security labs, and audit writeups.	Build two polished projects: one DeFi protocol integration and one Solana or L2 system.	Apply/propose consistently, perform mock interviews, audit public code, and contribute fixes/documentation.

References and Further Reading

Solidity Documentation: <https://docs.soliditylang.org/>

Foundry Documentation: <https://getfoundry.sh/>

OpenZeppelin Contracts: <https://docs.openzeppelin.com/contracts/5.x/>

OpenZeppelin Upgrades Plugins: <https://docs.openzeppelin.com/upgrades-plugins/>

Ethereum Developer Documentation: <https://ethereum.org/developers/>

Solana Documentation: <https://solana.com/docs>

Solana Anchor Documentation: <https://solana.com/docs/programs/anchor>

Uniswap Documentation: <https://docs.uniswap.org/>

Aave Developer Documentation: <https://aave.com/docs/>

Chainlink Documentation: <https://docs.chain.link/>

The Graph Documentation: <https://thegraph.com/docs/>

Version note: Blockchain tooling changes quickly. Before deploying production code, verify current compiler, dependency, chain, protocol, and deployment documentation. Pin versions and test upgrades rather than relying on course examples alone.